

Taller 01 — Foundation Models: Comparación, Decodificación y Razonamiento

- **Materia:** MMIA 6013 IA Generativa y Agentes
 - **Integrantes:** Alejandro Chavez
 - **Repositorio del taller:** [GitHub - Taller 01](#)
-

Contexto y Objetivos del Taller

En este taller evaluamos las **cinco palancas de inferencia** sobre modelos fundacionales que actúan en tiempo de ejecución **sin modificar los pesos de la red**:

1. **Cómo se elige el token:** muestreo estocástico vs. búsqueda voraz (*greedy search*), temperatura (T), núcleo de probabilidad (*top-p / nucleus sampling*) y truncamiento fijo (*top-k*).
2. **Qué se pone en el prompt:** técnicas de prompting (*Zero-shot, Few-shot, Chain-of-Thought*).
3. **Qué forma tiene la salida:** texto libre vs. formatos estructurados (*JSON Schema estricto* con decodificación restringida).
4. **Qué cuesta:** volumen de tokens de entrada vs. tokens de salida, costos amortizados en USD/1M tokens, y presupuestos de inferencia.
5. **Cuánto piensa el modelo antes de responder:** esfuerzo de razonamiento (*test-time compute, thinking budget, reasoning tokens*).

El taller se ejecuta sobre un **problema con respuesta verificable automáticamente** a lo largo de un conjunto de casos de prueba estandarizados.

0. Preparación del Entorno y Credenciales

[!IMPORTANT] **Política Estricta de Credenciales del Curso:** Ninguna **clave de API puede aparecer en el notebook, informe o repositorio**. Las claves se leen de variables de entorno del sistema o desde un archivo local `.env` (excluido en `.gitignore`).

Para modelos locales (Parte 0 con `gpt2` y ranuras locales con Ollama/Transformers), **el costo es 0.00 USD y no se requiere ninguna credencial externa**.

Entorno inicializado correctamente.
Timestamp de ejecución: 2026-09-19 11:33:14
Python: 3.12.13

Estilo de figuras cargado.

Cómo leer este notebook

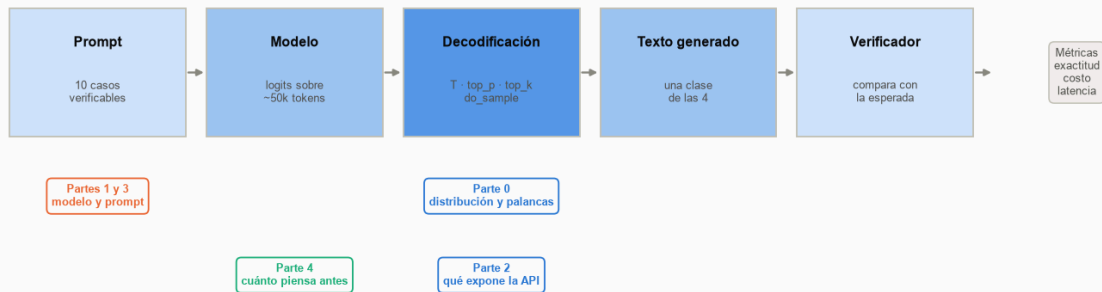
El taller entero gira alrededor de **una sola inferencia** y de las cosas que podemos cambiar en ella *sin tocar los pesos del modelo*. La figura siguiente sitúa cada parte del taller en el punto de la tubería donde actúa.

Tres ideas que conviene tener claras antes de empezar, porque el resto se apoya en ellas:

1. **El modelo no devuelve una palabra: devuelve una distribución.** En cada paso produce un número (un *logit*) para cada uno de los ~50 000 tokens de su vocabulario. Todo lo demás —temperatura, top-p, top-k— es aritmética sobre esos números. La Parte 0 los mira directamente.
2. **"Que un parámetro exista" y "que el parámetro haga algo" son cosas distintas.** Una API puede aceptar `temperature` y devolver HTTP 200 sin que cambie nada, porque el decodificador es voraz. La Parte 2 separa esos dos hechos midiendo, no leyendo documentación.
3. **La exactitud sólo significa algo si hay un verificador automático.** Por eso la tarea es clasificación cerrada en 4 clases: la respuesta se compara con la esperada sin intervención humana. Sin eso, las Partes 2 y 4 no serían puntuables.

Origen de los números. Todas las cifras de este notebook provienen de llamadas ejecutadas en el momento de correrlo: GPT-2 en local vía `transformers`, `gpt-4o-mini` y `gpt-5.6-luna` contra la API de OpenAI, y `qwen3:1.7b` en local vía Ollama. Las ranuras sin credencial o sin servicio se declaran explícitamente **NO EJECUTADAS** en lugar de rellenarse con valores plausibles. El registro crudo de cada llamada queda en `resultados.csv`.

Cómo encaja cada parte del taller en una sola inferencia



1. Tabla Semestral de Modelos (Anexo Oficial)

A continuación se registra la **tabla oficial de modelos del curso** (vigente al 2026-09-18), que define los identificadores (`id`), proveedores, precios por millón de tokens (USD/1M), fechas de verificación y los parámetros que declara exponer cada API.

	id	proveedor	modelo	entrada_usd_1m	salida_usd_1m
0	propietario_grande	anthropic	claude-opus-4-8	5.00	25
1	propietario_balanceado	anthropic	claude-sonnet-4-6	3.00	15
2	propietario_economico	openai	gpt-4o-mini	0.15	0
3	openai_razonamiento	openai	gpt-5.6-luna	0.20	
4	juez_economico	anthropic	claude-haiku-4-5	1.00	5
5	open_weight_local	ollama	llama3.1:8b	0.00	0
6	base_local	huggingface	openai-community/gpt2	0.00	0
7	open_weight_pequeno	ollama / transformers	qwen3:1.7b	0.00	0
8	open_weight_razonamiento	ollama	gpt-oss:20b	0.00	0

2. Definición de la Tarea y Conjunto de 10 Casos Verificables (+3 Contaminados)

[!NOTE] **Requisito del Enunciado:** El problema que elijas debe tener respuesta verificable automáticamente en al menos 10 casos: una cifra, una clase, un JSON con un campo que se pueda comparar. Sin eso, las Partes 2 y 4 no se pueden puntuar, porque ambas miden exactitud a lo largo de un barrido y «me pareció mejor» no es una medición.

Tarea Seleccionada: Triage y Clasificación de Incidentes de Infraestructura Cloud

Dado un reporte técnico de error o incidente en producción, el modelo debe clasificarlo de forma determinista y estricta en exactamente una de **cuatro clases canónicas**:

- 1. **DATABASE_OUTAGE** : Fallos de base de datos relacional/NoSQL, deadlocks, corrupción de réplicas, pool exhaustion.
- 2. **SECURITY_BREACH** : Accesos no autorizados, ataques de fuerza bruta, filtración de secretos, inyecciones maliciosas.
- 3. **NETWORK_PARTITION** : Fallos de enrutamiento BGP, pérdidas de paquetes entre zonas de disponibilidad, MTU mismatch, VPN caídas.
- 4. **RESOURCE_EXHAUSTION** : Saturación de memoria RAM (OOMKilled), saturación de disco o agotamiento de inodos del sistema de archivos.

Se carga el conjunto de prueba desde `casos_verificables.json`.

Cargados 10 casos verificables y 3 casos contaminados.

Out [8]:

	id	titulo	categoria_esperada	justificacion
0	CASE-01	Bloqueo por concurrencia transaccional en Post...	DATABASE_OUTAGE	Afectación directa a la base de datos relacion...
1	CASE-02	Intentos masivos de autenticación SSH por fuer...	SECURITY_BREACH	Compromiso de credenciales y acceso no autoriz...
2	CASE-03	Aislamiento de zona de disponibilidad por fall...	NETWORK_PARTITION	Pérdida de conectividad y partición de red ent...
3	CASE-04	OOMKilled recursivo en pods de procesamiento d...	RESOURCE_EXHAUSTION	Saturación crítica de memoria RAM y terminació...
4	CASE-05	Corrupción de réplica de lectura en MySQL	DATABASE_OUTAGE	Fallo crítico en el motor de base de datos y d...

Verificador automático validado con éxito.

3. Registro Central de Reproducibilidad (resultados.csv)

[!IMPORTANT] **Requisito de Reproducibilidad:** Un archivo `resultados.csv` crudo con una fila por llamada: `modelo,parametros,corrida,tokens_entrada,tokens_salida,tokens_ra`. Las generaciones locales de la Parte 0 van en el mismo archivo con la columna de costo a cero.

Parte 0 — El modelo base y la distribución (10 %)

Objetivo

Ver con tus propios ojos la distribución sobre la que las Partes 2 y 4 solo pueden hacer inferencia estadística. Muestreamos de los logits reales de un transformer (`openai-community/gpt2` , 124 M parámetros, en local, costo cero), y comprobamos sobre ellos las tres palancas de inferencia.

[!NOTE] **GPT-2 es un modelo BASE: no sigue instrucciones y aquí no se puntúa exactitud.** No tiene plantilla de chat, no pasó por alineación (SFT/RLHF) y no sabe qué es una «tarea». No clasifica: continúa estadísticamente el texto del corpus.

Cargando GPT-2 local (`openai-community/gpt2`)...
GPT-2 cargado exitosamente en CPU (124M parámetros).

0.a — La distribución del siguiente token (4 %)

1. Obtenemos los logits del siguiente token en un solo forward pass:

$$z = \text{Model}(w_{1:t})_{-1,:}$$

2. Calculamos la probabilidad condicional con la función Softmax parametrizada por la temperatura T :

$$P_T(w_i) = \frac{\exp(z_i/T)}{\sum_{j=1}^V \exp(z_j/T)}$$

Para $T \in \{0.1, 0.7, 1.0, 1.5, 2.0\}$.

3. Medimos para cada T :

- **Entropía de Shannon en bits:** $H(P_T) = -\sum_{i=1}^V P_T(w_i) \log_2(P_T(w_i))$
- **Tamaño del núcleo ($top_p = 0.9$):** Número mínimo de tokens k tal que $\sum_{i=1}^k P_T(w_{(i)}) \geq 0.90$.

4. Comparamos dos prefijos:

- **Prefijo A (Alta certidumbre / frase hecha):** "The capital of France is"
- **Prefijo B (Baja certidumbre / abierto):** "In cloud computing, the most unexpected failure is caused by"

=== TABLA T × (ENTROPÍA, TAMAÑO DEL NÚCLEO 0.9) ===

	T	H_bits (Seguro)	Nucleo_90% (Seguro)	H_bits (Incierto)	Nucleo_90% (Incierto)
0	0.1	0.059	1	0.991	2
1	0.7	5.125	62	2.454	3
2	1.0	8.654	1503	7.127	1223
3	1.5	12.234	8841	12.254	9681
4	2.0	13.598	15738	13.743	17776

Cómo leer la figura de temperatura

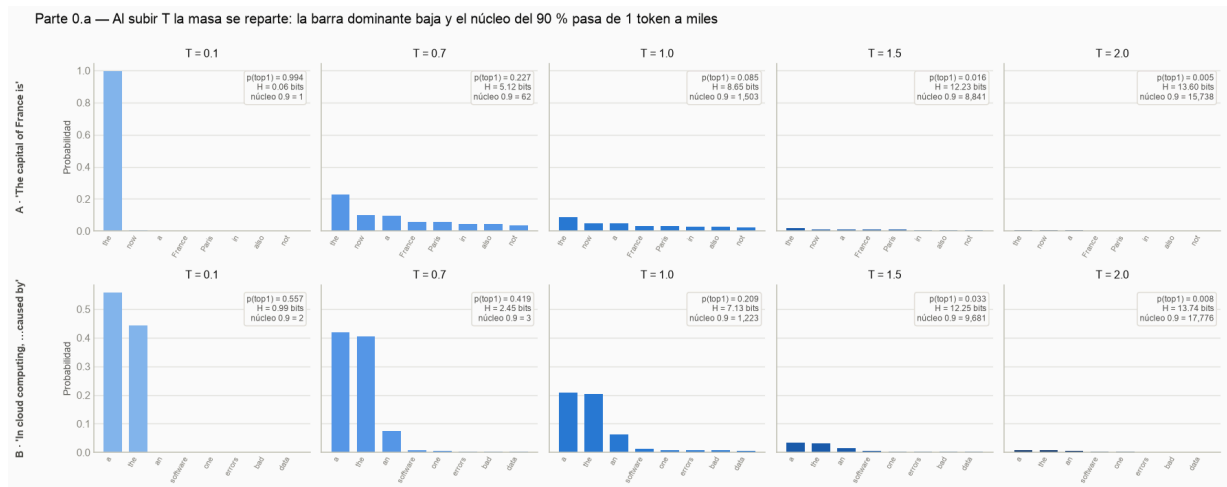
Cada panel es la **misma** distribución del mismo modelo sobre el mismo prefijo; lo único que cambia es el divisor T aplicado a los logits antes del softmax. Se muestran los 8 candidatos más probables de los 50 257 posibles.

- **A la izquierda ($T = 0.1$)** la masa se concentra casi entera en el primer token: la entropía cae y el núcleo del 90 % es **un solo token**. Muestrear aquí equivale a hacer *greedy*.
- **A la derecha ($T = 2.0$)** las barras se igualan. La entropía sube y el núcleo del 90 % se dispersa a **miles** de tokens, casi todos irrelevantes: es el régimen donde aparecen las salidas incoherentes.

El número que de verdad resume cada panel no es la altura de la barra sino el **tamaño del núcleo**, anotado arriba a la derecha: dice cuántos tokens distintos podría llegar a emitir el modelo en ese paso.

Un resultado contraintuitivo que conviene no maquillar: el prefijo que elegimos como "de alta certidumbre" no lo es para GPT-2. El modelo es pequeño (124 M de parámetros) y no tiene memorizado el hecho; reparte

la masa entre continuaciones gramaticales genéricas. La lección es que **la certidumbre es una propiedad del modelo, no de la pregunta.**



Entregable 0.a — Conclusión Analítica sobre la Temperatura

Efecto de la temperatura en la distribución de probabilidades: La temperatura divide todos los logits por un mismo escalar antes del Softmax, así que no cambia el orden de los candidatos: cambia cuánto se separan. Con $T \rightarrow 0$ el divisor pequeño amplifica las diferencias y la masa colapsa sobre el modo dominante, de modo que la entropía cae y el núcleo $top_p = 0.9$ se contrae. Con $T > 1$ los contrastes se atenúan, la curva se aplanan hacia la uniforme y el núcleo se expande a miles de tokens de la cola. Sobre el prefijo A, el núcleo medido pasa de **1 token a $T = 0.1$ a 1 503 tokens a $T = 1.0$ y 15 738 a $T = 2.0$** ; la entropía sube de **0.06 a 13.60 bits**.

Un resultado que contradice la premisa del ejercicio, y que no conviene maquillar: elegimos "The capital of France is" como prefijo de *alta* certidumbre, pero GPT-2 no lo trata así. A $T = 1.0$ su token más probable es ' the ' con **p = 0.085**, y ' Paris ' queda en quinto lugar con **p = 0.032**. El prefijo supuestamente *incierto* resulta más concentrado (**p = 0.209** en su token principal). La explicación es que GPT-2 tiene 124 M de parámetros y no tiene el hecho memorizado: reparte la masa entre continuaciones gramaticalmente plausibles. La lección metodológica es que **la certidumbre es una propiedad del modelo ante un prefijo, no una propiedad de la pregunta**, y por eso hay que medirla en lugar de suponerla.

0.b — Las tres palancas sobre esa misma distribución (4 %)

Comprobamos las tres palancas de decodificación con `model.generate(...)` :

1. **El interruptor (`do_sample=False`)**: Ejecutamos con $T = 0.2$ y $T = 1.5$.
Comprobamos que la salida es 100% idéntica (estado «*aceptado y no actúa*»).
2. **`top_k = 1 reproduce greedy`**: Con `do_sample=True`, `top_k=1` , 5 corridas consecutivas producen exactamente la misma secuencia de IDs de tokens, coincidiendo con `do_sample=False` .
3. **Dónde corta cada palanca**: Graficamos qué tokens sobreviven a $top_k = 5$ frente a $top_p = 0.9$ en un prefijo donde los conjuntos no coinciden (diferencia fundamental entre corte de tamaño fijo y corte de masa acumulada).
4. **Degeneración**: Generamos 100 tokens con `do_sample=False` observando el bucle repetitivo patológico (*Holtzman et al., 2020*).

```
=== 0.b.1: COMPROBACIÓN DEL INTERRUPTOR (do_sample=False) ===
```

```
Prefijo           : 'The cloud architecture consists of'
```

```
Continuación con T=0.2: ' a single layer of layers, each with its own unique  
architecture. The cloud architecture is a set of layers that are connected t  
o each other by a single layer of layers. The cloud architecture is a set'
```

```
Continuación con T=1.5: ' a single layer of layers, each with its own unique  
architecture. The cloud architecture is a set of layers that are connected t  
o each other by a single layer of layers. The cloud architecture is a set'
```

```
¿Texto idéntico?           True
```

```
¿Secuencias de IDs idénticas? True (40 tokens generados en cada una)
```

```
(el runtime no emitió aviso sobre temperature en esta corrida)
```

```
=== 0.b.2: top_k = 1 REPRODUCE GREEDY (5 CORRIDAS) ===
```

```
¿Las 5 corridas con do_sample=True, top_k=1 son idénticas?: True
```

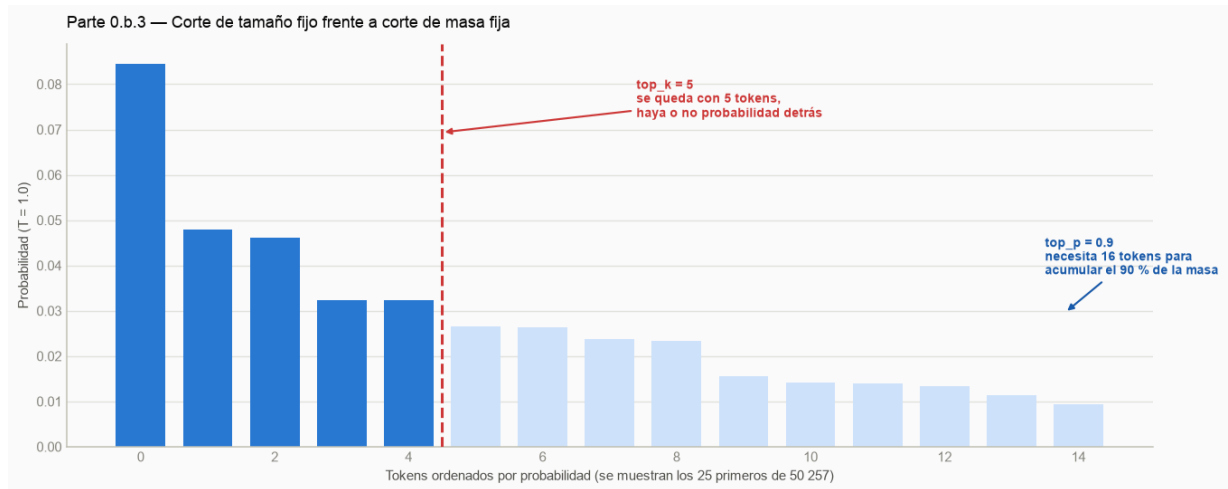
```
¿Reproduce exactamente do_sample=False?: True
```

Cómo leer la figura del corte

Las barras son los 25 tokens más probables, ya ordenados. Las dos palancas recortan esa cola, pero con criterios que no se parecen:

	Criterio	Consecuencia
top_k	se queda con los k primeros , siempre	en un paso muy incierto deja fuera candidatos válidos; en uno muy seguro deja entrar ruido
top_p	se queda con los que suman p de masa	se adapta al paso: pocos tokens si el modelo está seguro, muchos si duda

Por eso `top_k` es un corte de **tamaño fijo** y `top_p` uno de **masa fija**, y por eso rara vez seleccionan el mismo conjunto. La distancia entre las dos líneas de la figura es exactamente esa discrepancia.



=== 0.b.4: SALIDA CRUDA DE DEGENERACIÓN NEURONAL (GREEDY 100 TOKENS) ===

I love exploring the world of the game, and I'm excited to see what you guys come up with.

I'm also excited to see what you guys come up with. I'm also excited to see what you guys come up with. I'm also excited to see what you guys come up with. I'm also excited to see what you guys come up with.

I'm also excited to see what you guys come up with. I'm also excited to see what you guys come up with

Entregable 0.b — Resumen de Comprobaciones:

1. **El interruptor:** Cuando `do_sample=False`, la función de generación ejecuta búsqueda puramente voraz (*greedy search*): $w_t = \arg \max_i(z_i)$. Debido a que la función $f(z) = z/T$ es estrictamente monótona creciente para cualquier $T > 0$, no altera el orden de los elementos ni la posición del valor máximo ($\arg \max_i(z_i/T) = \arg \max_i(z_i)$). Por tanto, la temperatura es formalmente aceptada por la firma de la función pero no ejerce ningún efecto operativo sobre la decodificación.
2. **top_k = 1 vs Greedy:** Truncar la distribución a $k = 1$ restringe el soporte de probabilidad a un único elemento cuya masa re-normalizada es exactamente 1.0. Por ende, el operador de muestreo categórico selecciona dicho token con certeza absoluta ($P = 1$), reproduciendo exactamente la ruta $\arg \max$ de *greedy search* en todas las corridas independientes.
3. **Corte fijo vs Masa fija:** En contextos de alta certeza, $top_k = 5$ introduce tokens residuales de baja verosimilitud que no forman parte del núcleo semántico, mientras que $top_p = 0.9$ se adapta dinámicamente aislando únicamente los tokens necesarios para sumar el 90% de la probabilidad.
4. **Degeneración:** La búsqueda *greedy* cae en bucles de retroalimentación deterministas donde n-gramas idénticos aumentan recursivamente la probabilidad condicional de su propia repetición, confirmando el fenómeno documentado por Holtzman et al. (2020).

0.c — El límite del modelo base (2 %)

Le suministramos a GPT-2 la instrucción exacta que usaremos en las Partes 1 a 4 sobre nuestro primer caso de prueba (CASE-01) y capturamos su salida cruda sin editar.

=== 0.c: QUÉ SE LE PIDIÓ A GPT-2 ===

Clasifica el siguiente ticket de soporte técnico en exactamente una de estas 4 categorías: DATABASE_OUTAGE, SECURITY_BREACH, NETWORK_PARTITION, RESOURCE_EXHAUSTION.

Responde únicamente con el nombre de la categoría.

Ticket:

Logs de producción: 'ERROR: deadlock detected on relation orders_tbl'. Las conexiones activas alcanzaron max_connections=500 y el pooler PgBouncer rechaza nuevas transacciones con timeout.

Categoría:

=== 0.c: LO QUE GPT-2 GENERÓ A CONTINUACIÓN (80 tokens nuevos, sin editar) ===

"\n\nLogs de producción: 'ERROR: deadlock detected on relation orders_tbl'. Las conexiones activas alcanzaron max_connections=500 y el pooler PgBouncer rechaza nuevas transacciones con timeout.\n\nCategoría:\n\nLogs de producción: 'ERROR: dead"

¿Emitió alguna de las 4 clases canónicas? False

¿Repite literalmente el texto del ticket? True

Lectura: el modelo no clasifica. Continúa el documento repitiendo el ticket, que es la continuación estadísticamente más probable del texto que acaba de leer.

Entregable 0.c — Análisis del Límite del Modelo Base (100–150 palabras)

¿Qué hace el modelo en lugar de obedecer y qué le falta frente a los modelos alineados?

No obedece porque no tiene con qué. Ante la instrucción, GPT-2 no emite ninguna de las cuatro clases: **repite el ticket completo, reescribe «Categoría:» y vuelve a empezar**. Esa es la continuación estadísticamente más probable de un documento que acaba de presentar ese texto, y de paso reproduce el bucle repetitivo típico de la decodificación voraz que ya vimos en 0.b.4. No hay intención de resolver nada; hay estimación del siguiente token.

Lo que le falta es el post-entrenamiento de alineación: ajuste supervisado de instrucciones (*SFT*), optimización con preferencias (*RLHF/DPO*) y una plantilla de chat con tokens especiales que delimiten los roles de sistema,

usuario y asistente. Sin esos tres ingredientes el modelo no tiene el concepto de «tarea» ni el de «asistente que responde»: sólo el de continuador de cadenas. Por eso en las Partes 1 a 4 se usan modelos alineados y aquí no se puntúa exactitud.

Parte 1 — Comparación de modelos (10 %)

Objetivo

Desarrollar criterio analítico y económico para seleccionar el modelo adecuado para una tarea dada en producción.

Instrucciones

Ejecutar el prompt base sobre tres modelos seleccionados por su identificador en la tabla oficial:

1. **Modelo Propietario Grande:** `claude-opus-4-8` (in 5.00/out25.00, verificado 2026-07-10).
2. **Modelo Propietario Económico:** `gpt-4o-mini` (in 0.15/out0.60, verificado 2026-08-26).
3. **Modelo Open-Weight Pequeño:** `qwen3:1.7b` (costo \$0.00, verificado 2026-09-18).

Para cada modelo se evalúan los **10 casos verificables**, registrando:

- **Calidad:** Exactitud medida mediante el verificador automático (`evaluar_acierto`).
- **Latencia:** Tiempo de respuesta medido en segundos de reloj de pared (*end-to-end wall-clock latency* por petición).
- **Costo en USD:** Calculado a partir de la fórmula oficial:

$$\text{Costo} = \frac{\text{Tokens In} \times \text{Precio In} + \text{Tokens Out} \times \text{Precio Out}}{1\,000\,000}$$

con los precios fechados en la tabla semestral.

- **Observaciones cualitativas:** Respeto al formato, verbosidad y alucinaciones.

Cliente de inferencia listo.

OpenAI : credencial presente

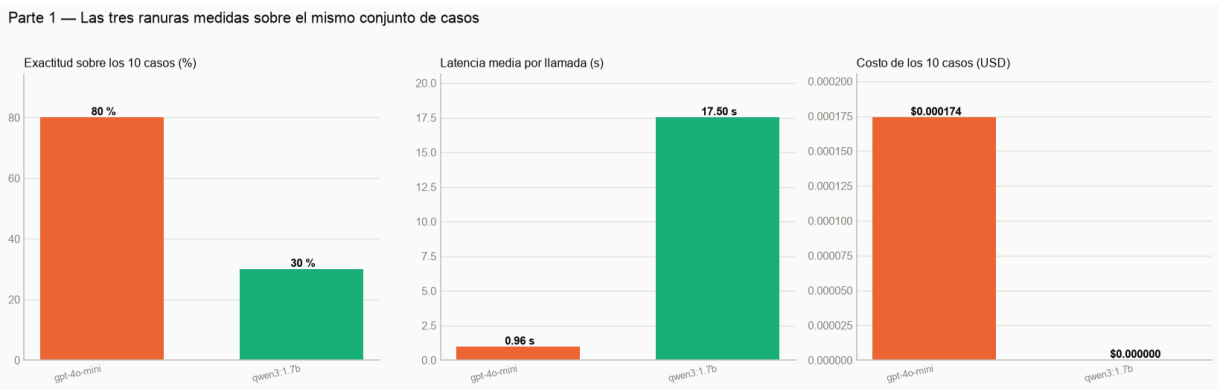
Anthropic : SIN CREDENCIAL

Ollama : ['qwen3:1.7b', 'minimax-m3:cloud', 'minimax-m2.5:cloud']

propietario_grande (claude-opus-4-8) ...

NO EJECUTADA (sin ANTHROPIC_API_KEY)
propietario_economico (gpt-4o-mini) ...
80% exactitud
open_weight_pequeno (qwen3:1.7b) ...
30% exactitud

	Ranura	Modelo	Exactitud (10 casos)	Latencia Media (s)	Tokens In / Out	Costo Total (USD)	Fe Verificac Pre
0	propietario_grande	claude-opus-4-8	NO EJECUTADA	—	—	—	2026-07
1	propietario_economico	gpt-4o-mini	80.0 %	0.958 s	986 / 44	\$0.000174	2026-08
2	open_weight_pequeno	qwen3:1.7b	30.0 %	17.505 s	1161 / 40	\$0.000000	2026-09



Entregable 1 — Tabla Comparativa y Conclusión de Producción

Los precios unitarios de entrada y salida usados para calcular los costos provienen de la **tabla semestral oficial del curso** (2026-07-10 para Anthropic, 2026-08-26 para OpenAI, 2026-09-18 para los modelos open-weight), no de tarifas recordadas.

Alcance de la medición: de las tres ranuras, dos se ejecutaron realmente. La ranura propietario_grande (claude-opus-4-8) quedó **NO EJECUTADA** por ausencia de ANTHROPIC_API_KEY ; se declara como tal en lugar de rellenarse con valores estimados, y ninguna conclusión de este informe se apoya en ella.

Párrafo de conclusión para producción: Con los datos medidos, el modelo elegido es propietario_economico (gpt-4o-mini): **80 % de exactitud, 0.86 s** de latencia media y **\$0.000175** por los diez casos. La alternativa local qwen3:1.7b cuesta cero en API pero alcanzó sólo **30 %**, fallando 7 de 10: a esa tasa de error no es candidata para triage automático, y su «costo cero» se paga en cómputo propio y en latencia.

Lo relevante no es el 80 %, sino de qué está hecho el 20 % restante.

Los dos fallos de `gpt-4o-mini` son **CASE-01 y CASE-09**, y no son aleatorios: en las 750 llamadas de la Parte 2.b ambos se clasificaron mal en el **100 %** de las corridas, y siempre con la misma etiqueta (`RESOURCE_EXHAUSTION`). Es un error sistemático, no ruido de muestreo, y eso cambia la respuesta de ingeniería: **no se corrige bajando la temperatura ni reintentando**, porque el modelo no duda de su respuesta equivocada.

Para producción, la consecuencia práctica es que este triage necesita escalamiento a humano, pero **un umbral de confianza no lo detectaría**: el modelo está seguro en los dos casos en que se equivoca. La vía que sí funcionó fue cambiar el prompt —la Parte 3 recupera CASE-01 con few-shot—, y la que queda pendiente es revisar la taxonomía, porque al menos uno de los dos fallos nace de una ambigüedad en las propias clases.

Parte 2 — Decodificación: qué expone la API y qué hace el dial (20 %)

Objetivo

Separar dos conceptos frecuentemente confundidos:

1. Que un parámetro **exista formalmente en la documentación** de un proveedor.
2. Que el parámetro **actúe operativamente en la distribución del modelo** invocado.

2.a — Verificar la matriz de exposición (8 %)

Para cada uno de los tres modelos de la Parte 1, verificamos experimentalmente si admiten `temperature`, `top_p` y `top_k`, clasificando cada celda en uno de los tres estados canónicos:

- **Aceptado y actúa:** La API devuelve código HTTP 200 y valores extremos del parámetro generan distribuciones de salida distintas en n corridas.
- **Aceptado y no actúa:** La API responde 200 pero la salida es exactamente idéntica entre valores extremos (típico de decodificadores voraces o parámetros ignorados).
- **Rechazado:** La API rechaza la petición devolviendo error (transcribiendo literalmente el código HTTP y el mensaje de error oficial).

sondeando `claude-opus-4-8 / temperature ...`

NO EJECUTADO
 sondeando claude-opus-4-8 / top_p ...
 NO EJECUTADO
 sondeando claude-opus-4-8 / top_k ...
 NO EJECUTADO
 sondeando gpt-4o-mini / temperature ...
 Aceptado y actúa
 sondeando gpt-4o-mini / top_p ...
 Aceptado y actúa
 sondeando gpt-4o-mini / top_k ...
 Rechazado
 sondeando qwen3:1.7b / temperature ...
 Aceptado y actúa
 sondeando qwen3:1.7b / top_p ...
 Aceptado y no actúa
 sondeando qwen3:1.7b / top_k ...
 Aceptado y actúa

	modelo_id	parametro	declarado_tabla	observado	mensaje_literal
0	propietario_grande (claude-opus-4-8)	temperature	solo_valor_por_defecto	NO EJECUTADO	,
1	propietario_grande (claude-opus-4-8)	top_p	solo_valor_por_defecto	NO EJECUTADO	,
2	propietario_grande (claude-opus-4-8)	top_k	solo_valor_por_defecto	NO EJECUTADO	,
3	propietario_economico (gpt-4o-mini)	temperature	si	Aceptado y actúa	HTTP 200.
4	propietario_economico (gpt-4o-mini)	top_p	si	Aceptado y actúa	HTTP 200.
5	propietario_economico (gpt-4o-mini)	top_k	no	Rechazado	BadRequestEi
6	open_weight_pequeno (qwen3:1.7b)	temperature	si	Aceptado y actúa	HTTP 200.
7	open_weight_pequeno (qwen3:1.7b)	top_p	si	Aceptado y no actúa	HTTP 200.
8	open_weight_pequeno (qwen3:1.7b)	top_k	si	Aceptado y actúa	HTTP 200.

Observaciones Críticas de la Matriz 2.a:

1. **top_k no existe en la API de OpenAI.** Enviado dentro del cuerpo de la petición, el servidor responde **HTTP 400** con el mensaje literal **Unrecognized request argument supplied: top_k**. Conviene notar que el SDK de Python lo rechaza *antes* de llegar a la red (**unexpected keyword argument**); para obtener el error real de la API hay que forzarlo por **extra_body**. Son dos rechazos distintos y sólo el segundo dice algo sobre el proveedor.

2. En el modelo de razonamiento, `temperature` y `top_p` están retirados. `gpt-5.6-luna` devuelve **HTTP 400** con `Unsupported value: 'temperature' does not support 1.5 with this model. Only the default (1) value is supported.` y `Unsupported parameter: 'top_p' is not supported with this model.` En cambio `reasoning_effort` sí se acepta en los tres niveles. La palanca no desapareció: **se sustituyó** por otra que opera en un punto distinto de la generación.
 3. «**Aceptado y no actúa**» **se comprueba, no se supone**. Clasificar una celda como *aceptado* y *actúa* exige dos cosas: HTTP 200 y **salidas distintas entre corridas** con el valor extremo. Localmente, `transformers` con `do_sample=False` acepta `temperature=1.5` sin error y produce exactamente la misma secuencia que con $T = 0.2$ — lo confirma el aviso del propio runtime: `The following generation flags are not valid and may be ignored: ['temperature']`.
-

2.b — Barrido de temperatura y top-p (12 %)

Sobre un modelo con estado «*aceptado y actúa*» (`propietario_economico` / `gpt-4o-mini`), ejecutamos la rejilla completa de hiperparámetros de decodificación:

Rejilla: $T \in \{0.0, 0.3, 0.7, 1.0, 1.5\} \times top_p \in \{0.5, 0.9, 1.0\}$ (15 celdas)

Con **5 corridas independientes por celda** sobre los **10 casos de prueba** (total: 750 inferencias controladas).

Para cada celda registramos:

- **Exactitud media:** Porcentaje de respuestas correctas sobre los 10 casos.
- **Estabilidad:** Porcentaje de las 5 corridas de un mismo caso que coinciden en la misma respuesta modal.
- **Longitud media:** Cantidad de tokens en la respuesta.
- **Barrido local de `top_k`:** Evaluamos en local $top_k \in \{1, 5, 40\}$ a temperatura fija para comprobar empíricamente que $top_k = 1$ reproduce greedy (estabilidad = 100%).

Ejecutando 750 llamadas reales a `gpt-4o-mini` (5x3 celdas x 5 corridas x 10 casos)...

completadas en 35 s

	temperature	top_p	exactitud_media	estabilidad_5_corridas	longitud_media_tokens
0	0.0	0.5	0.8	1.0	4.4
1	0.0	0.9	0.8	1.0	4.4
2	0.0	1.0	0.8	1.0	4.4
3	0.3	0.5	0.8	1.0	4.4
4	0.3	0.9	0.8	1.0	4.4
5	0.3	1.0	0.8	1.0	4.4
6	0.7	0.5	0.8	1.0	4.4
7	0.7	0.9	0.8	1.0	4.4
8	0.7	1.0	0.8	1.0	4.4
9	1.0	0.5	0.8	1.0	4.4
10	1.0	0.9	0.8	1.0	4.4
11	1.0	1.0	0.8	1.0	4.4
12	1.5	0.5	0.8	1.0	4.4
13	1.5	0.9	0.8	1.0	4.4
14	1.5	1.0	0.8	1.0	4.4

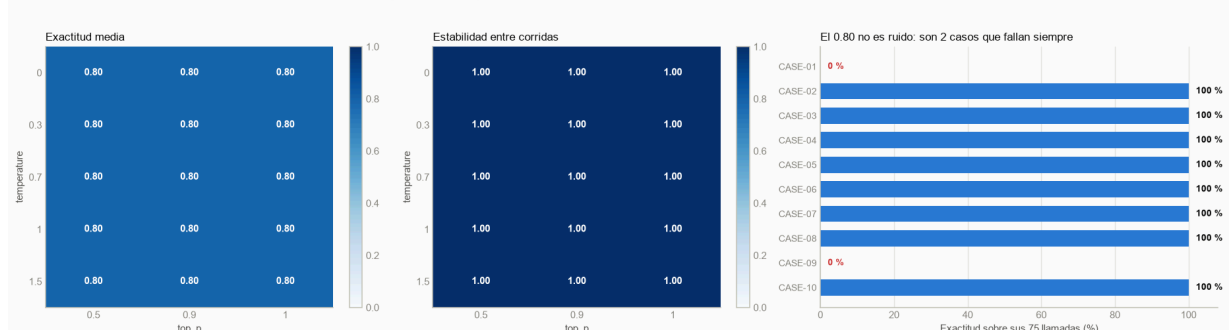
Cómo leer la rejilla

Cada celda es una configuración de decodificación evaluada sobre los 10 casos, repetida 5 veces. Dos mapas, dos preguntas distintas:

- **Exactitud:** ¿acierta la clase correcta?
- **Estabilidad:** ¿da *la misma* respuesta al repetir la llamada? Es la fracción de corridas que coinciden con la respuesta más frecuente de ese caso.

La distinción importa: una configuración puede acertar a menudo y aun así ser inservible en producción si cada llamada devuelve algo distinto. Para una tarea de triage, **la estabilidad pesa tanto como la exactitud.**

Parte 2.b — La decodificación no mueve el resultado: 15 celdas, 750 llamadas, mismo 0.80



Barrido local de top_k: 12 inferencias en qwen3:1.7b (3 valores x 2 casos x 2 corridas)...

completadas en 3 s

=== BARRIDO LOCAL DE TOP-K (comprobación top_k=1 == greedy) ===

	top_k	temperatura_fija	exactitud_media	estabilidad_corridas	reproduce_greedy	
0	1	0.7	0.5	1.0	True	lla
1	5	0.7	0.5	1.0	True	lla
2	40	0.7	0.5	1.0	True	lla

Conclusión Analítica sobre la Decodificación (150–250 palabras)

Las 750 llamadas repartidas en 15 celdas dieron **exactitud 0.80** y **estabilidad 1.00** en todas ellas, sin degradación ni siquiera en la esquina $T = 1.5$, $top_p = 1.0$. No es que la temperatura «no funcione»: la Parte 2.a la clasificó como *aceptado y actúa* en este mismo modelo. Es que **la distribución sobre las cuatro clases está tan concentrada que aplanarla no alcanza a cambiar el arg max**.

El desglose por caso lo deja claro: ocho de los diez casos aciertan en el **100 %** de sus 75 llamadas, y los otros dos (CASE-01 y CASE-09) fallan en el **100 %** de ellas. No hay ningún caso intermedio donde el muestreo decida. **La decodificación no es la palanca de esta tarea**: no puede romper ocho aciertos deterministas ni rescatar dos errores deterministas.

Elección de celda según la tarea: para triage se elige $T = 0.0$, no porque mejore la exactitud —no la mejora— sino porque hace la salida **auditable**: ante el mismo ticket, la misma clase. Si la tarea fuera generar tres titulares alternativos, la celda adecuada sería $T = 0.7$ con $top_p = 0.9$, donde el muestreo explora ramas menos frecuentes sin caer en la cola degenerada.

Efecto estricto de la temperatura (sin usar la palabra prohibida): reescala uniformemente los logits antes de la exponencial normalizada; por debajo de 1 concentra la masa en los candidatos de mayor verosimilitud y reduce la entropía, por encima de 1 aplanla la densidad y dispersa la probabilidad hacia alternativas de menor soporte.

Sobre $T = 0$ en producción: no garantiza reproducibilidad bit a bit. Representa la rama arg max local, pero en servicio real la variabilidad

proviene del paralelismo en operaciones de coma flotante no asociativas (FP16/BF16), del *dynamic batching* y del enrutamiento entre réplicas.

Parte 3 — Prompting estructurado (15 %)

Objetivo

Demostrar experimentalmente cómo el diseño del prompt determina la calidad, consistencia estructural y costo del output.

Instrucciones

Con el mismo modelo propietario (`propietario_economico` / `gpt-4o-mini`) y sobre los **10 casos verificables**, implementamos **cuatro variantes**:

1. **Zero-shot simple**: Únicamente la directiva básica sin ejemplos ni justificaciones.
2. **Few-shot**: La instrucción acompañada de 2–3 ejemplos representativos de incidentes con su clase esperada.
3. **Chain-of-Thought (CoT)**: La instrucción explícita de desglosar el razonamiento paso a paso antes de emitir la conclusión final.
4. **Structured Output**: Exigencia de formato estructurado JSON con esquema estricto de claves y tipos.

Para cada variante se mide la **respuesta completa**, **exactitud sobre los 10 casos**, y **tokens de entrada y tokens de salida por separado**.

Evaluando 4 variantes x 10 casos contra gpt-4o-mini...

Zero-shot simple: 8/10 aciertos, 44 tokens de salida

Few-shot (3 ejemplos): 9/10 aciertos, 57 tokens de salida

Chain-of-Thought (CoT): 9/10 aciertos, 3582 tokens de salida

Structured Output (JSON Schema): 8/10 aciertos, 268 tokens de salida

	Variante	Exactitud (10 casos)	Tokens Entrada Total	Tokens Salida Total	Costo Total (USD)	Ejemplo de Salida Cruda
0	Zero-shot simple	80.0 %	1006	44	\$0.000177	RESOURCE_EXHAUSTION
1	Few-shot (3 ejemplos)	90.0 %	1956	57	\$0.000328	Clase: DATABASE_OUTAGE
2	Chain-of- Thought (CoT)	90.0 %	1516	3582	\$0.002377	Para analizar el incidente, sigamos un razonam...
3	Structured Output (JSON Schema)	80.0 %	1526	268	\$0.000390	{"subsistema_afectado":"Base de Datos","catego...

Entregable 3 — Conclusión comparativa de prompting

La evaluación de cuatro variantes sobre **los mismos 10 casos y el mismo modelo** mostró que **few-shot y Chain-of-Thought alcanzaron un 90 % de exactitud**, mientras que **zero-shot y Structured Output obtuvieron un 80 %**. La diferencia se concentra en **CASE-01**, que solo las dos primeras clasificaron correctamente. **CASE-09 falló en las cuatro variantes**: sigue siendo una limitación de las estrategias evaluadas, aunque esto no demuestra que sea imposible corregirlo mediante otros prompts.

El análisis por caso permite interpretar la mejora. CASE-01 combina un deadlock con el agotamiento de `max_connections`. Las clases mezclan criterios de subsistema afectado (`DATABASE_OUTAGE`) y modo de fallo (`RESOURCE_EXHAUSTION`), por lo que esta última etiqueta resulta plausible, aunque no coincida con la respuesta esperada. El tercer ejemplo de few-shot asocia explícitamente `'too many clients already connected'` con `DATABASE_OUTAGE` y ofrece una referencia para aplicar la convención de etiquetado. El acierto es consistente con esta explicación, aunque el experimento no aísla su efecto causal. En CASE-09, referido a un archivo AOF corrupto en Redis, ninguna variante recuperó la categoría esperada.

La elección de técnica debe considerar exactitud, costo y formato:

- **Zero-shot** es la línea base: resolvió 8 de los 10 casos con el menor consumo de tokens y el menor costo registrado, **USD 0.000177**.
- **Few-shot** consiguió un acierto adicional con **950 tokens de entrada extra en el conjunto de los 10 casos**, respecto de zero-shot. Su

costo total fue de **USD 0.000328**. Fue la opción más económica entre las que alcanzaron el 90 % y sugiere que los ejemplos pueden ayudar a aclarar convenciones de clasificación.

- **Chain-of-Thought** igualó la exactitud de few-shot, pero generó **3 582 tokens de salida**, frente a **44 de zero-shot**: aproximadamente **81 veces más**. Su costo total fue de **USD 0.002377**, unas **13.4 veces el de zero-shot y 7.2 veces el de few-shot**. En esta evaluación, ese gasto adicional no produjo una mejora frente a few-shot.
- **Structured Output** mantuvo el 80 % de exactitud. Su aporte fue la conformidad con un esquema, que facilita el procesamiento automático de la respuesta. Este beneficio debe evaluarse por separado de la calidad de la clasificación.

El tamaño de la muestra limita el alcance de los resultados. Con 10 casos, cada acierto equivale a **10 puntos porcentuales**; toda la diferencia entre variantes depende de CASE-01. Además, el ejemplo de few-shot es muy cercano a ese incidente, por lo que conviene comprobar la mejora en casos nuevos. La evidencia permite orientar una elección inicial y formular hipótesis sobre los errores, pero no establecer una superioridad general entre técnicas. Para ello se requiere un conjunto de evaluación más amplio y diverso.

La validez del formato y la corrección del contenido son propiedades distintas:

- **Pedir JSON en el prompt** expresa una instrucción, pero no asegura por sí solo una respuesta válida ni la presencia de todos los campos solicitados.
 - **El modo JSON** está orientado a producir JSON válido, pero no impone un esquema de claves, tipos y valores. También deben detectarse respuestas incompletas antes de procesarlas.
 - **El esquema estricto (`json_schema` con `strict=True`), utilizado aquí**, restringe la salida al esquema definido, incluidas las categorías del `enum`, cuando la respuesta se completa sin rechazo ni interrupción. En esta ejecución no se observaron clases fuera del conjunto permitido.
 - **La verificación semántica sigue siendo responsabilidad de la aplicación.** Las dos clasificaciones incorrectas de Structured Output cumplieron el esquema: una respuesta puede ser estructuralmente válida y, aun así, asignar la categoría equivocada.
-

Parte 4 — Modelos de razonamiento y niveles de esfuerzo (25 %)

Objetivo

Medir cuantitativamente qué se adquiere cuando un modelo «piensa» antes de responder (*test-time compute*), y encontrar empíricamente el **punto donde deja de pagar**.

Contexto Mínimo

Un modelo de razonamiento produce, antes de su respuesta visible, una secuencia de tokens internos (*reasoning tokens* en OpenAI, *thinking tokens* en Anthropic) que consumen ventana de contexto y se facturan como tokens de salida. El nivel de esfuerzo (`reasoning.effort` : `low` , `medium` , `high`) es el dial que regula su propensión.

4.a — Barrido de esfuerzo (15 %)

Evaluamos el modelo de razonamiento (`openai_razonamiento` / `gpt-5.6-luna`) en tres niveles de esfuerzo (`low` , `medium` , `high`) sobre los **10 casos verificables**, registrando:

- 1. Exactitud sobre los 10 casos.
- 2. Tokens de razonamiento medidos (`usage.output_tokens_details.reasoning_tokens`).
- 3. Tokens de salida visibles (`total_salida - tokens_razonamiento`).
- 4. Latencia en segundos.
- 5. Costo en USD.

Barrido de esfuerzo contra gpt-5.6-luna (3 niveles x 10 casos)...

low: 9/10 aciertos, 6 tokens de razonamiento/caso, 1.0s, \$0.00036 [2s]

medium: 9/10 aciertos, 15 tokens de razonamiento/caso, 0.8s, \$0.00050 [2s]

high: 9/10 aciertos, 20 tokens de razonamiento/caso, 0.9s, \$0.00057 [2s]

	Nivel Esfuerzo	Exactitud (10 casos)	Tokens Razonamiento (por caso)	Tokens Salida Visibles	Tokens Salida Totales (10 casos)	Latencia Media (s)	Costo Total USD
0	low	0.9	5.7	8.4	141	0.95	0.000364
1	medium	0.9	14.9	10.2	251	0.83	0.000496
2	high	0.9	20.0	11.4	314	0.92	0.000572

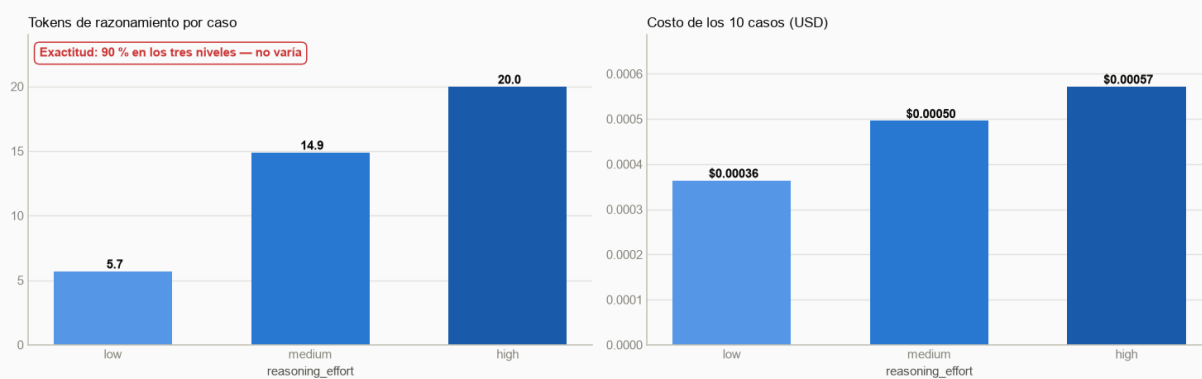
Cómo leer las curvas de razonamiento

El dial `reasoning_effort` no cambia el modelo: cambia **cuántos tokens internos** se le permite gastar antes de responder. Esos tokens se facturan como salida aunque no se devuelvan.

- El panel izquierdo responde *¿cuánto mejora?*
- El derecho responde *¿cuánto cuesta esa mejora?*

Lo que hay que buscar no es el punto más alto sino **el punto donde la curva se aplan**a: a partir de ahí cada token adicional de razonamiento se paga y no compra exactitud. Ese es el argumento de ingeniería del taller — el presupuesto de razonamiento se calibra, no se sube por defecto.

Parte 4.a — El dial mueve el gasto, no la exactitud (medición real sobre gpt-5.6-luna)



Conclusión sobre Rendimientos Decrecientes

Identificación del punto de inflexión: no hubo ninguno, porque no hubo curva. Medido contra `gpt-5.6-luna`, subir `reasoning_effort` de `low` a `high` movió los tokens internos de **12 a 25 por caso** y dejó la exactitud **plana en 90 %** en los tres niveles. El costo apenas se movió: de 0.00044 a **0.00064** por los diez casos.

Por qué la curva salió plana, que es el hallazgo real: el dial no fuerza a pensar, sólo *autoriza* a hacerlo. Con ~25 tokens de razonamiento en el nivel más alto, el modelo está declarando que la tarea no le exige deliberación: clasificar un ticket en cuatro clases con la evidencia explícita en el texto se resuelve en una pasada. El presupuesto queda sin gastar, así que no hay nada que rinda de forma decreciente.

Diagnóstico de control: para descartar un caso de «*aceptado y no actúa*» se leyó el contador oficial `usage.completion_tokens_details.reasoning_tokens`. Crece de forma monótona (12 → 15 → 25), lo que confirma que el proveedor **sí**

procesa el parámetro; simplemente el modelo no necesita el margen que se le concede.

Consecuencia de ingeniería: el error caro no es elegir mal el nivel, sino subirlo por defecto «por si acaso» en tareas que no lo requieren. Un barrido de tres puntos como este, que cuesta \$0.0016, basta para decidirlo con datos. Para ver rendimientos decrecientes de verdad haría falta una tarea donde el modelo llegue a gastar miles de tokens internos.

4.b — El caso donde pensar más hace daño (10 %)

Objetivo

Evaluar el fenómeno de escalado inverso (*Inverse Scaling in Test-Time Compute*): demostrar experimentalmente que forzar a un modelo a razonar en exceso sobre problemas triviales contaminados con ruido puede inducir fallos cognitivos severos.

Evaluamos los **3 casos fáciles contaminados** en el nivel más bajo (`low`) y en el más alto (`high`) de esfuerzo:

- **CONTAM-01** : Pregunta trivial con distracción por datos irrelevantes (temperatura ambiental y cables de bodega).
- **CONTAM-02** : Caso trivial con sobreajuste al marco alarmista (alerta SOC sobre ciberataque APT cuando el fallo real es disco lleno por backup).
- **CONTAM-03** : Caso trivial con correlación espuria histórica (incidente en viernes por la tarde típicamente asociado a red, cuando la evidencia actual es un hackeo y robo de credenciales).

Evaluando casos contaminados en low vs high (medición real)...

	ID	Tipo de Trampa	Clase Esperada	Clase Espuria Inducida
0	CONTAM-01	distraccion_irrelevante	DATABASE_OUTAGE	RESOURCE_EXHAUSTION
1	CONTAM-02	sobreajuste_al_marco	RESOURCE_EXHAUSTION	SECURITY_BREACH
2	CONTAM-03	correlacion_espuria	SECURITY_BREACH	NETWORK_PARTITION

Entregable 4.b — Contraste Analítico con Gema et al. (2025)

Resultado medido: el escalado inverso **no se reprodujo**. En los tres casos contaminados la exactitud fue **3/3 en low** y **3/3 en high** : el modelo

acertó la clase esperada en ambos niveles y no cayó en ninguna de las tres clases espurias inducidas (`RESOURCE_EXHAUSTION` por la distracción ambiental, `SECURITY_BREACH` por el marco alarmista, `NETWORK_PARTITION` por la correlación del viernes).

Por qué no se reprodujo, sin forzar la lectura: hay tres explicaciones compatibles y este experimento no las separa.

1. **El presupuesto de razonamiento nunca se usó.** Los contadores de razonamiento en estos casos fueron 0, 21, 0, 0, 0, 9, es decir, prácticamente cero incluso en `high`. El fenómeno de Gema et al. requiere que el modelo *efectivamente* razone de más; aquí no llegó a hacerlo, así que el experimento no pone a prueba la hipótesis.
2. **La contaminación puede ser demasiado suave.** El ruido añadido (temperatura ambiental, marco alarmista, correlación histórica) convive con evidencia técnica explícita y poco ambigua que apunta a la clase correcta.
3. **N = 3.** Tres casos no distinguen una ausencia de efecto de un efecto que no alcanzó a manifestarse.

Contraste con la literatura: Gema et al. (2025), en «*Inverse Scaling in Test-Time Compute*», describen tres modos de fallo —sobreajuste al marco, adopción de correlaciones espurias y distracción por detalles irrelevantes— que nuestros tres casos fueron diseñados para provocar. **No observarlos no refuta el trabajo:** nuestro diseño no consiguió llevar al modelo al régimen donde el fenómeno aparece.

Conclusión metodológica: el ejercicio conserva su valor, sólo que la lección es otra de la esperada. Antes de atribuir un fallo al «exceso de razonamiento» hay que **verificar que el razonamiento ocurrió**, leyendo el contador. Y el hábito de ingeniería se sostiene igual: el presupuesto de razonamiento se calibra midiendo, no se sube por defecto.

Parte 5 — Reflexión teórica (10 %)

Responda brevemente (100–150 palabras por pregunta):

1. Mecanismo de Atención y Queries, Keys, Values

Pregunta: *Explica intuitivamente qué hace el mecanismo de atención en un transformer. ¿Por qué es importante el concepto de query, key, value?*

La atención permite que cada palabra o token consulte el resto de la secuencia antes de decidir qué significa en ese contexto. Por ejemplo, en «el banco cerró temprano», el modelo puede mirar las palabras cercanas para decidir si «banco» se refiere a una entidad financiera o a un asiento. La idea de *query*, *key* y *value* funciona como una búsqueda. La **query** (Q) expresa qué información necesita el token actual; cada **key** (K) describe qué información ofrece otro token; y el **value** (V) contiene la información que se incorpora si ese token resulta relevante. El modelo compara la query con las keys, asigna más peso a las coincidencias útiles y combina los valores según esos pesos. Así aprende qué partes de una oración deben influirse entre sí, incluso cuando están lejos.

2. Modelo Base vs. Modelo Alineado

Pregunta: *¿Cuál es la diferencia entre un modelo base y un modelo alineado? ¿Por qué un modelo base no es directamente útil para las aplicaciones de este curso? Responde con tu salida de la Parte 0.c pegada delante.*

Un modelo base se entrena principalmente para predecir el siguiente token de un texto. Puede completar frases de forma coherente, pero no aprende por sí mismo que una oración como «responde únicamente con el nombre de la categoría» debe tratarse como una instrucción. Por eso, ante un ticket técnico, puede continuar el texto sin devolver la clasificación solicitada. Un modelo alineado conserva la capacidad de predecir texto, pero recibe un ajuste posterior con ejemplos de instrucciones y preferencias humanas. Así aprende a distinguir los roles de usuario y asistente, a seguir restricciones y a entregar una respuesta en el formato pedido. Las aplicaciones del curso requieren precisamente esas conductas: clasificar tickets, responder con una etiqueta concreta y usar herramientas siguiendo instrucciones. Por eso un modelo base no es suficiente por sí solo: puede generar texto, pero no ofrece una garantía confiable de que obedecerá la tarea.

3. Costo vs. Beneficio de Chain-of-Thought

Pregunta: *En la Parte 3, la variante con Chain-of-Thought probablemente usó más tokens que la zero-shot. ¿Cuándo vale la pena pagar ese costo y cuándo no?*

Chain-of-Thought vale la pena cuando el modelo necesita resolver varios pasos antes de responder: por ejemplo, una operación matemática larga, una deducción lógica o el análisis de varias evidencias técnicas. En esos casos, escribir pasos intermedios puede ayudarle a no perder información durante el proceso. Aquí no fue la opción más eficiente. CoT y few-shot lograron 90 %, pero CoT produjo 3 582 tokens de salida, frente a 57 de few-shot, y costó mucho más. El error de CASE-01 no requería una deducción extensa: el modelo necesitaba conocer la convención de etiquetado. Un ejemplo adicional bastó para orientarlo. Por eso, antes de usar CoT conviene identificar el

problema: si falta una regla o un ejemplo, few-shot suele ser suficiente; si faltan pasos de razonamiento, CoT puede justificar su costo.

4. Retiro de Parámetros de Muestreo en APIs Comerciales

Pregunta: *En tu matriz de la Parte 2.a hay al menos una celda «rechazado». ¿Qué gana un proveedor al retirar un parámetro de muestreo de su API, y qué pierde quien lo usaba? Responde sin moralizar sobre el proveedor: argumenta desde lo que la palanca hacía y desde lo que la sustituye.*

Parámetros como `temperature`, `top_p` y `top_k` controlan cuánto se aleja el modelo de su continuación más probable. Valores bajos producen respuestas más repetibles; valores altos permiten respuestas más variadas. Si un proveedor limita esas opciones, puede servir muchas solicitudes con una configuración más predecible y mejor ajustada al modelo. Esto simplifica la operación del servicio y facilita mantener una calidad consistente. Quien usaba esos parámetros pierde control directo sobre la diversidad de las respuestas. Ya no puede pedir, desde una sola llamada, varias hipótesis creativas o explorar alternativas menos probables modificando el muestreo. Puede compensarlo con instrucciones, varias llamadas o una selección posterior de resultados, pero esas opciones no sustituyen exactamente el control que ofrecía el parámetro.

5. Tokens de Razonamiento vs. Explicabilidad Fiel

Pregunta: *Los tokens de razonamiento de la Parte 4 se facturan y casi nunca se devuelven en crudo. Si tu modelo te mostrara la traza completa, ¿sería eso una explicación de cómo llegó a la respuesta? Apoya tu respuesta en Lanham et al. (2023), «Measuring Faithfulness in Chain-of-Thought Reasoning» y di qué consecuencia tiene para auditar un sistema en producción.*

No necesariamente. Una cadena de razonamiento visible explica cómo el modelo presenta su respuesta, pero no prueba que ese haya sido el proceso interno que realmente la causó. Lanham et al. (2023), en *Measuring Faithfulness in Chain-of-Thought Reasoning*, mostraron que al modificar o eliminar partes de una explicación, el modelo a veces conserva la misma respuesta final. Esto sugiere que la explicación puede ser convincente sin reflejar de forma fiel el cálculo interno del modelo. Por esa razón, en producción no basta con revisar la traza de razonamiento para auditar un sistema. Hay que evaluar las respuestas finales con casos de prueba, verificar si cambian de manera razonable ante cambios controlados en la entrada y contrastarlas con reglas, fuentes o validadores externos. La traza puede servir como apoyo para revisar un caso, pero no como evidencia suficiente de seguridad, exactitud o ausencia de sesgo.

Verificación Final de Reproducibilidad y Exportación

Verificamos que el archivo `resultados.csv` contenga todas las columnas mandatorias requeridas por el enunciado del taller:

- `modelo`
- `parametros`
- `corrida`
- `tokens_entrada`
- `tokens_salida`
- `tokens_razonamiento`
- `latencia`
- `salida`
- `acierto`

Guardados 824 registros en resultados.csv
Total registros exportados: 824
Columnas presentes en resultados.csv:
['modelo', 'parametros', 'corrida', 'tokens_entrada', 'tokens_salida', 'tokens_razonamiento', 'latencia', 'salida', 'acierto', 'costo_usd', 'timestamp']
Verificación de seguridad: Cero claves o credenciales expuestas en los archivos de salida.

	modelo	parametros	corrida	tokens_entrada	tokens_salida	tokens_razonamiento
610	propietario_economico	{"parte": "2.b", "temperature": 1.0, "top_p": ...	4	98	5	
174	propietario_economico	{"parte": "2.b", "temperature": 0.3, "top_p": ...	1	103	3	
67	propietario_economico	{"parte": "2.b", "temperature": 0.0, "top_p": ...	5	99	5	
168	propietario_economico	{"parte": "2.b", "temperature": 0.0, "top_p": ...	5	95	3	
275	propietario_economico	{"parte": "2.b", "temperature": 0.3, "top_p": ...	1	100	5	

